

Bird Identification Using Neural Networks

CASE STUDY—DATA MINING CUP

Lecturer:
Steffen Maletz

Author

Vineet Murlidhar Daishri 257827

August 31, 2025

TU DORMTUND

Contents

List of Figures	III
List of Tables	III
List of Acronyms	IV
1 Introduction	1
1.1 Motivation and Objectives	1
1.1.1 Motivation	1
1.1.2 Objectives	2
1.2 Problem Statement	2
2 Data	3
2.1 Data Exploratory Analysis	4
2.2 Data Pre-processing	6
2.2.1 Audio Loading, Resampling and Segmentation.....	6
2.2.2 Mel Spectrogram Conversion	6
3 Methods	8
3.1 Short-Time Fourier Transform (STFT)	8
3.2 Resample	9
3.3 Train-test splitting and Data batching	9
3.4 ConvNeXt	9
3.5 ECA-Net	11
3.6 Focal Loss	11
3.7 AUC-ROC.....	12
3.8 Adam	13
3.8.1 Learning Rate (α)	14
3.9 Softmax.....	15
3.10 F1, Precision and Recall	15
3.11 Sigmoid	16
3.12 K-fold cross validation.....	16

3.13 Data Augmentation.....	18
4 Model Evaluation	19
4.1 ConvNeXt Model	19
4.1.1 AUC-ROC Diagnostic	19
4.1.2 Precision-Recall Diagnostic	20
4.1.3 F1-score Diagnostics.....	21
4.2 ECA-Net Model (External Training)	22
5 Inference: Weighted Blend	23
5.1 Weighted Blending.....	24
6 Conclusion	25
6.1 Limitations	25
6.2 Potential Improvements	26
References	27

List of Figures

2.1	(a) Shows the right-skewed distribution of clip lengths in seconds, and (b) Illustrates the multimodal distribution of quality ratings.	4
2.2	Distribution of training recordings across all 206 bird species	5
2.3	Different species and labels	5
2.4	Before and after implementing preprocessing on data	7
2.5	The structure of the 4-D input tensor after Batching	7
3.1	ConvNeXT model architecture	10
3.2	ECA module overview (adapted from [53]; see also [35]).....	11
4.1	AUC-ROC for bird species	19
4.2	Precision vs. recall per class; marker size $\propto$ support, color $\propto$ F1-score.	20
4.3	eCDF of per-class F1-scores (206 species). Points are shown for every 10th class .	21
4.4	Train vs. validation AUC-ROC per epoch for the ECA-Net model.....	22

List of Tables

2.1	Structure of the train.csv metadata file	3
5.1	Example for final predictions CSV file for 5 s chunk	24

List of Acronyms

Adam	Adaptive Moment Estimation
AUC-ROC	Area Under the Curve, Receiver Operating Characteristic
CNN	Convolutional Neural Network
ConvNet	Convolutional Neural Network
ConvNeXT	Contemporary Architecture for Convolutional Neural Networks
ECA	Efficient Channel Attention
ECA-Net	Efficient Channel Attention network
ECA-NFNet	Efficient Channel Attention Normalizing Free Network
eCDF	Empirical Cumulative Distribution Function
FLOP	Floating Point Operations
FN	False Negative
FP	False Positive
FPR	False Positive Rate
GELU	Gaussian Error Linear Unit
IAvH-CSA	Instituto Alexander von Humboldt Colección de Sonidos Ambientales
n_fft	Fast Fourier Transform
ONNX	Open Neural Network Exchange
ResNet	Residual Neural Network
SE	Squeeze and Excitation Network
SNR	Signal to Noise Ratio
STFT	Short Time Fourier Transform
Swin	Shifted Window Transformer
TN	True Negative
TP	True Positive
TPR	True Positive Rate
XCL	Xeno Canto

1 Introduction

The use of automated biodiversity monitoring serves as an important tool in conservation biology and ecological research to observe animal populations along with their habitats without causing damage. Bioacoustics functions as a practical bird species monitoring technique which extracts essential data about distribution patterns, behavioral characteristics, and population diversity through sound analysis of vocalizations [49].

The large audio streams from environmental sensors are time-consuming to process and must be analyzed by skilled human experts. Such a lag in ecological data processing constitutes a severe bottleneck to prompt conservation choices. This project explicitly tackles this challenge by creating a completely automated framework for recognizing bird species from continuous audio data till classification for birds. The developed machine learning system can detect 206 different bird species from processed 5-second audio clips, successfully addressing issues of heavy background noise, overlapping calls, and large bird variations in calls.

A deep learning framework employs CNN ensembles to reach this objective. The methodology relies on combining two models using weighted blending. The included models are ConvNeXT trained on the XCL dataset, covering 9000+ bird species from XCL and then fine-tuned on 206 bird species, and an ECA-NFNet model with post-processing with power adjustment for low rank. The developed system achieves improved **0.861** AUC-ROC score and generalization through Mel spectrogram conversion of raw audio and the combined feature extraction strengths of heterogeneous architectures. This report details the complete methodology starting from audio preprocessing including feature extraction, model architecture specifications, model training, ensembling strategy, and the final inference pipeline for deployment [27, 53].

1.1 Motivation and Objectives

1.1.1 Motivation

The motivation for this study arises from the urgent need for more efficient and up-scaled methods for monitoring biodiversity. Bird populations are among bio-indicators, that is, bird health and diversity reflect the condition of their ecosystems. Current bird survey methods, such as manual point counts, are labor-intensive, locally restricted, and wildlife-disturbing. Bioacoustic monitoring offers a passive, continuous, and non-invasive option, but it suffers from the enormous volume of data it can generate. It is impossible to analyze thousands of hours of sound by hand and this represents an intrinsic data-processing barrier that paralyzes effective conservation action in a timely manner [16].

1.1.2 Objectives

The overarching goal of this project is to build, deploy, and evaluate a high-performance deep-learning system that can automatically classify 206 bird species from continuous raw audio recordings. To realize this aim, this project defines three concrete technical objectives:

1. Build a solid ensemble model: The core technical objective is to create a powerful classification system by ensembling two distinct, state-of-the-art CNN architectures: ConvNeXT and ECA-NFNet [27, 53].
2. Implement an improved audio-to-image preprocessing pipeline: The primary goal for using image-based CNNs requires converting single-dimensional audio data into two-dimensional Mel spectrograms.
3. Demonstrates how a reliable and fast workflow can train and test bird-species classifiers in under 90 minutes using only a CPU and no internet. This is achieved through preprocessing, batching, and ensembling more efficient, proving the system is ready for real-world use with limited resources.

1.2 Problem Statement

The research investigates the challenging process of detecting 206 different bird species automatically in uninterrupted real-world audio files. The main challenge exists in building a computational model that splits audio data into 5-second segments for independent species probability scoring [49].

The proposed solution needs to overcome multiple essential technical obstacles. The model needs exceptional resistance against background noise which originates from natural elements such as wind, rain, insects and human-generated sounds. The system requires exceptional performance in classification because real-world soundscapes commonly present simultaneous vocalizations of multiple species. The model tends to solve this complexity by separating overlapping acoustic cues while distinguishing between species that produce similar vocalizations [31].

The model demands precise feature extraction to differentiate between species while avoiding misclassifications for accurate identification. The system must provide exact temporal localization which enables it to identify species occurrences within each 5-second segment. The output will be a structured file that reports these probability scores for all segments, offering a comprehensive understanding of the bird species present in the audio recording [44].

2 Data

The dataset used in this work is from the BirdCLEF 2025 competition and consists of a large dataset of bird, amphibians, mammals and insect sounds were curated from XCL, iNaturalist, and IAvH-CSA sound archive, a community-driven database of bird sounds from across the globe. It is organized to best support the training and testing of models for real-world bioacoustic monitoring and consists of training data, test data, and additional metadata files. The main task is to classify which of the 206 potential bird species are present in any given 5-second window of an audio file [22, 55, 19, 20].

Table 2.1: *Structure of the train.csv metadata file*

Column	Description	Datatype	Example
<code>primary_label</code>	The main bird species present in the recording.	String	<code>amekes</code> (American Kestrel)
<code>secondary_labels</code>	A list of other species present, if any.	List	<code>['blujay', 'norcar']</code>
<code>type</code>	The type of vocalization (for example, call, song).	List	<code>['call', 'song']</code>
<code>filename</code>	The unique name of the audio file.	String	<code>amekes/XC12345.ogg</code>
<code>rating</code>	A quality rating for the recording from 1.0 to 5.0.	Float	<code>4.5</code>

The dataset includes the following components:

Training Audio (`train_audio`): The directory contains thousands of short audio clips that are stored in Ogg Vorbis format. The audio files contain primary bird vocalizations together with background noise and occasional secondary species calls. The `train_audio/` directory contains **206** subdirectories that correspond to target classes and are named after the numeric `primary_label` (taxon ID). The source recordings are stored in `.ogg` format within each directory. For example: `train_audio/22333/iNat292304.ogg`. The `primary_label` IDs from `taxonomy.csv` serve to link scientific/common names and function as identifiers in `sample_submission.csv`.

Training Metadata (`train.csv`): A CSV file that provides detailed labels and context for files in the `train_audio` directory. The structure of this file is crucial for supervised learning and is detailed in Table 2.1. **Test Soundscapes (`test_soundscapes`):** This directory contains the unlabeled audio files used for the final evaluation. These are continuous and multi-minute recordings that simulate a real-world monitoring scenario. The task is to generate predictions for each 5-second segment of these files. **Taxonomy Data (`taxonomy.csv`):** This file provides

the complete list of the 206 target species, mapping their short codes (for example `amekes`) to their scientific names and common names. **Sample Submission (`sample_submission.csv`):** This file dictates the required format for the final predictions, with a `row_id` column (e.g., `soundscape_123_5` for the first 5-second clip) and a column for each of the 206 species.

The metadata file contains extra columns which include geographic coordinates (`latitude`, `longitude`), `recording_date`, `author` and `license`. This information is useful for ecology, but this project deliberately excludes these fields from model training to ensure that the resulting models only use acoustic features. This design choice aims to improve generalization and avoid dependency on metadata that, in real-world deployments, may be unavailable or introduce irregularities [14].

2.1 Data Exploratory Analysis

To achieve the underlying nature and internal organization of the data set, an exhaustive exploratory data analysis (EDA) was performed before model development.

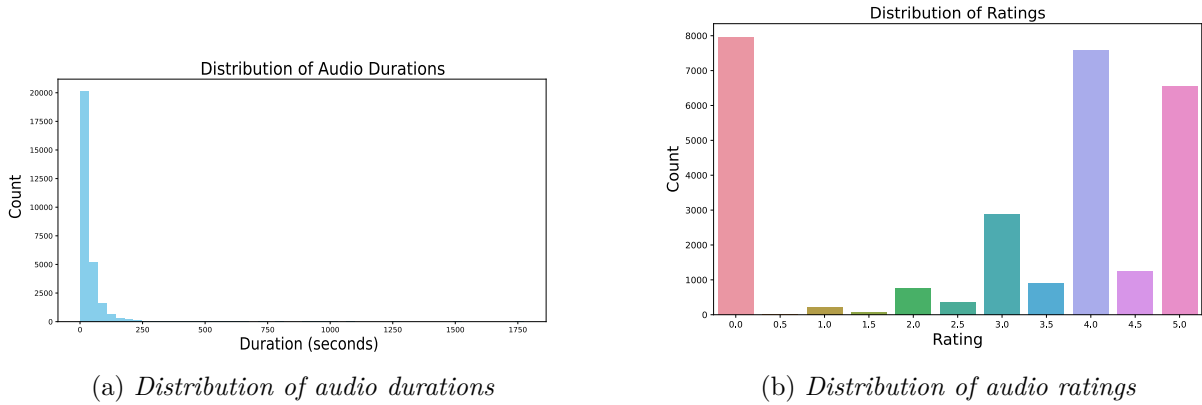


Figure 2.1: (a) Shows the right-skewed distribution of clip lengths in seconds, and (b) Illustrates the multimodal distribution of quality ratings.

The audio file structural analysis presented in figure 2.1 reveals wide variations between audio duration lengths and quality levels. The right-skewed histogram of audio durations in figure 2.1a reveals a strong rightward skew. The majority of recordings have short durations which cluster between 10 and 60 seconds. The distribution extends into a long tail which indicates that only a few recordings exceed 250 seconds in length. The recording quality analysis presented in figure 2.1b demonstrates multiple distinct distribution modes. The dataset contains numerous recordings that receive either no rating (0.0) or full ratings (4.0 and 5.0) which indicates the presence of both high-quality samples and extensive ineffective data. The system needs to learn generalization from this diverse training data to achieve useful performance on continuous multi-minute test soundscapes.

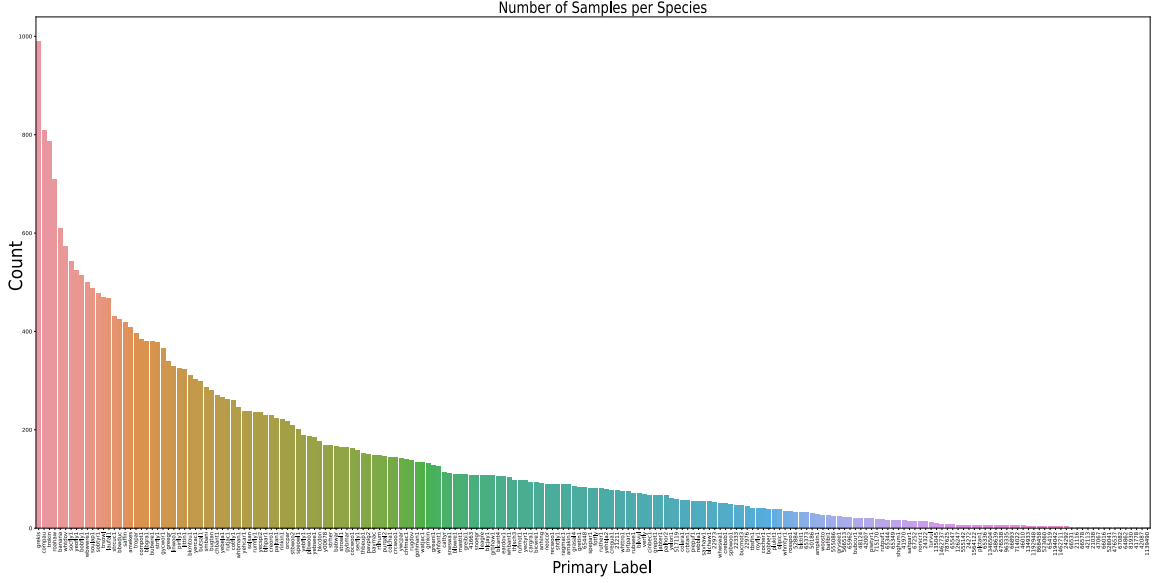


Figure 2.2: *Distribution of training recordings across all 206 bird species*

The number of examples spread over the 206 species, as shown in figure 2.2, reflects a large class imbalance that arises as a vital issue. The data have a long-tail distribution where some species are well-represented with nearly 1,000 recordings, and other species have very few examples. This imbalance can bias the model to frequent species. However, it requires special countermeasures to ensure that the model can accurately classify rare and underrepresented birds.

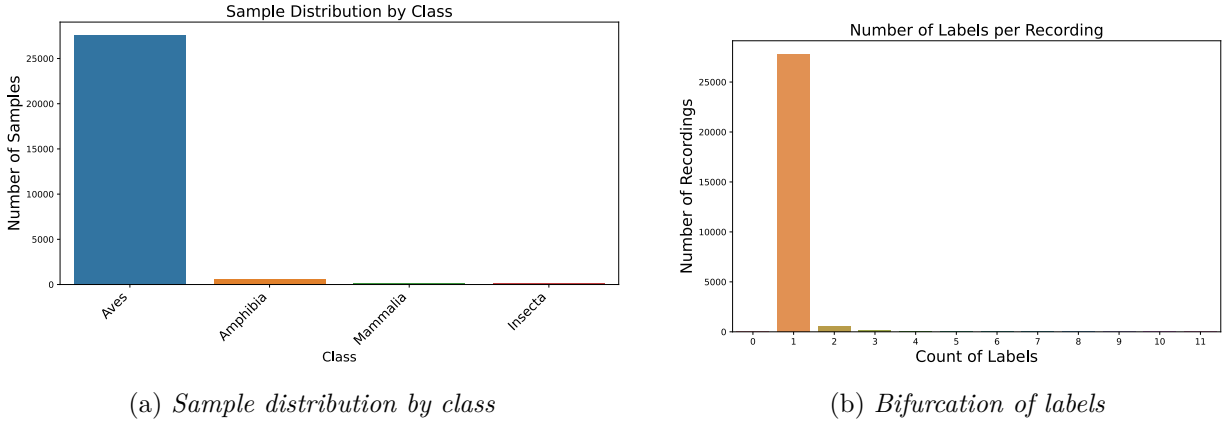


Figure 2.3: *Different species and labels*

Figure 2.3a shows that the dataset is heavily inclined toward bird species. The ‘Aves’ class makes up most of the samples, with nearly 28,000 entries. In contrast, ‘Amphibia’ has only about 600 to 700 samples, and both ‘Mammalia’ and ‘Insecta’ are even more underrepresented, each with fewer than 200 samples. This subset of species can increase class ambiguity due to overlapping decibel ranges and can potentially introduce classification issues that the model must address. In figure 2.3b, most recordings have just one label. Fewer than 800 samples have a secondary label, and cases with three or more labels are very rare. This strong imbalance suggests training the model for a multi-class problem instead of a multi-label one.

2.2 Data Pre-processing

The primary goal of the data pre-processing stage is to convert raw, variable-length audio recordings into a standardized, numerical format that can be effectively processed by deep learning models. Uniform processing methods were applied to all recordings since the dataset contained no missing data. The processing pipeline utilized standard audio processing libraries throughout the project.

2.2.1 Audio Loading, Resampling and Segmentation

The first operation in the pipeline is to load the audio files from the database. The recordings are all resampled to a common rate of 32 kHz regardless of the original sample rate. This standardization is crucial, as it makes the frequency and the time resolution the same for all samples so that the models cannot learn spurious features due to differences in recording quality. The test and training audio files are of varying lengths. To ensure inputs are equal for the model, the audio is divided into non-overlapping 5-second windows. The window length is standard for analyzing wildlife sound and captures most of the bird vocalization events in a suitable temporal context. For recordings in the test set, the same segmentation is applied sequentially to generate predictions by a 5-second window. In addition, no padding is applied to the raw audio. For instance, if a recording is 6 seconds long, the final 1 s is omitted to ensure that only complete 5-second segments are analyzed.

2.2.2 Mel Spectrogram Conversion

Once the audio is split, each 5-second waveform is converted to a Mel spectrogram 3-*D* tensor. This is a multi-step procedure designed to mimic the human hearing system by emphasizing the frequencies that are more audible to the human ear.

1. Spectrogram generation: First, a standard spectrogram is computed using STFT [36]. The resulting spectrogram was configured with a 1024 sample Fast Fourier Transform window and a hop length of 320 samples. The resulting *power* spectrogram, $S[n, k] = |X[n, k]|^2$, encodes the distribution of energy across frequencies over time and is represented as a 2D tensor of frequency bins by time frames. For a 5 s clip at 32 kHz with $n_{\text{fft}} = 1024$ and hop length $H = 320$, the number of time frames is

$$T = \left\lfloor \frac{5 \times 32000 - 1024}{320} \right\rfloor + 1 = 497 \quad \text{time frames}$$

2. Mel-scale transformation: The power spectrogram applies triangular filters to perform mel scale projection which transforms linear frequency data into logarithmic intervals

that correspond to human auditory pitch perception. The transform is configured with $n_{\text{mels}} = 128$, yielding a mel-spectrogram tensor of height 128 (one coefficient per mel band).

3. Conversion to Decibel (dB) Scale: The Mel power spectrogram was then converted to the logarithmic decibel (dB) scale. This squeezes the dynamic range of the audio, allowing quiet sounds to be visible and preventing loud sounds from dominating the representation. An 80 dB dynamic-range limit (`top_db=80`) is applied, constraining all values to be within 80 dB of the maximum. This ensures that components more than 80 dB below the peak are clipped, while low-energy features remain visible [41].
4. Normalization: The dB-scaled Mel spectrogram is finally normalized. This is a standard step in deep learning that normalizes the input features to have standardized the mean and variance (values are used from dataset statistics [41]). The normalization was performed with a pre-computed mean of -4.268 and a standard deviation of 4.569. Moreover, a singleton channel axis is added and a 3D tensor is produced to make the data set compatible while training (shown in figure 2.5).

The raw waveform (2.4a) presents a 5 s amplitude–time view that preserves intensity changes but conceals the spectral structure. The log–Mel spectrogram (2.4b) converts the signal into a 2D time–frequency map over 128 Mel bands; dB scaling with an 80 dB dynamic-range limit preserves low-energy content. The log–Mel spectrogram exposes ambient noise (broad, low-contrast beds from wind/insects/anthropogenic sources) and structured events (narrow, high-energy ridges). The highlighted region shows a compact, time-contiguous vocalization distinct from the background. Retaining realistic noise is beneficial for training, helping the classifier separate bird vocal patterns from non-vocal, broadband or stationary interference.

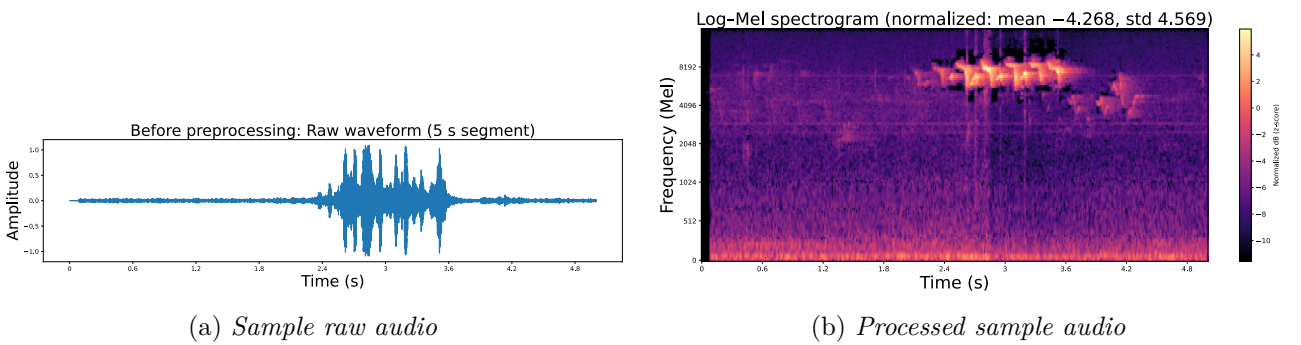


Figure 2.4: *Before and after implementing preprocessing on data*

$$\underbrace{\left(\underbrace{32}_{\text{Batch Size}} \times \underbrace{1}_{\text{Channels}} \times \underbrace{128}_{\text{Height (Mel Bins)}} \times \underbrace{497}_{\text{Width (Time Frames)}} \right)}_{\text{Dimensions of a single spectrogram}}$$

Figure 2.5: *The structure of the 4-D input tensor after Batching*

3 Methods

The research methodology includes three main steps that are the preparation of the dataset, the development of the model, and the evaluation. The raw inputs are standardized and transformed into learning-ready representations. Models are trained in designated training partitions and evaluated on testing data using complementary metrics to capture ranking quality (AUC-ROC). All experiments are performed with fixed random seeds with clearly defined splits to support reproducibility.

3.1 Short-Time Fourier Transform (STFT)

The discrete-time 5 s waveform $x[n]$ undergoes a 32kHz sampling to obtain the time-frequency characteristics. The signals undergo windowing through frames with a 1024 sample length and a 320 sample hop size. The Hann windowing function $w[m]$ is applied to each frame according to the following mathematical expression:

$$X[n, k] = \sum_{m=0}^{M-1} x[nH + m] w[m] e^{-2j\pi k m/M},$$

The frame index n ranges from 0 to $\lfloor \frac{L-M}{H} \rfloor$, while the frequency bin index k extends from 0 to $M-1$ and the total sample count is denoted by L . The power spectrogram $|X[n, k]|^2$ results from a magnitude-squared STFT (power = 2) which has dimensions $(M/2 + 1) \times T$ (`n_fft` = $M = 1024$). The power spectrogram results from a 5-second clip in which the time frames T can be computed as follows:

$$T = \left\lfloor \frac{5 \text{ s} \times 32000 - 1024}{320} \right\rfloor + 1 = 497$$

The number of frequency bins in the one-sided STFT becomes 513 (`n_stft` = $1024/2 + 1 = 513$). The power spectrogram has the following arrangement: $P \in \mathbb{R}^{513 \times 497}$ (frequency $\times$ time) [36]. The Mel-filterbank exists as a linear transformation between the mel frequencies and the frequency components through $W \in \mathbb{R}^{128 \times 513}$ (mel $\times$ frequency). Every time frame undergoes a matrix product operation to generate the $S_{\text{mel}} = W P \in \mathbb{R}^{128 \times 497}$ (mel $\times$ time) result. Each Mel band m and frame n gets processed through the following operation:

$$S_{\text{mel}}[m, n] = \sum_{k=1}^{513} W_{m,k} P_{k,n},$$

The matrix form replicates the operation of $\sum_k H_m(k) |X[n, k]|^2$. Adding a singleton channel gives the tensor shape $[1 \times 128 \times 497]$.

3.2 Resample

Multiple recording sources in the dataset use different native sample rates ranging from 8 kHz to 16 kHz to 48 kHz. All waveforms $x[n]$ that have their original sampling rates get resampled to a uniform rate of 32 Hz. The STFT and Mel-scale transforms will maintain identical sample spacing for all signals after resampling every clip to 32 kHz. The model cannot identify spurious signals because without this there will be a mismatch of time frequency resolution and spectrogram width. [49, 51].

3.3 Train-test splitting and Data batching

To avoid overfitting and for better generalization, the dataset is partitioned into a full collection of 5-second audio clips in non-overlapping training and testing subsets [10]. Let N be the total number of clips in the dataset. The formula is shown below:

$$N_{\text{train}} = \lfloor 0.8 N \rfloor, \quad N_{\text{test}} = 1 - N_{\text{train}},$$

Thus, allocating 80% of the data to training and the remaining 20% of the data for testing. Each subset is wrapped in a *DataLoader* to form mini-batches at training and inference time. A batch size of 32 is chosen to maintain the balance between stable gradient estimates and GPU memory limits. During training, the loader is configured with shuffling so that at each epoch, the order of the samples is randomized. This helps prevent the model from overfitting [30].

Within each batch, individual clips (after segmentation, resampling, conversion, and normalization) all have identical tensor dimensions $[1 \times 128 \times 497]$. The default tensor shape is $[B \times 1 \times 128 \times 497]$. This batch tensor is then supplied directly to the ConvNeXT classifier. This ensures that every example is seen during each epoch or evaluation pass.

3.4 ConvNeXt

The model architecture described in this section is shown in figure 3.1. The backbone is *ConvNeXT*, a modern, purely convolutional network that incorporates design choices inspired by Vision Transformers (for example patchified stem, depthwise separable blocks, LayerNorm, GELU) while retaining the inductive biases and efficiency of ConvNet. The classifier is built on ConvNeXT-Base, a modern convolutional architecture introduced by [27] as a ‘ConvNet for the 2020s’. ConvNeXT revisits a ResNet-style design and systematically modernizes it with ideas inspired by hierarchical Vision Transformers (for example, Swin) while remaining a pure convolutional network. Key elements include a non-overlapping patchify stem (a 4×4 convolution with stride 4), the use of large-kernel depthwise convolutions within blocks, LayerNorm in place

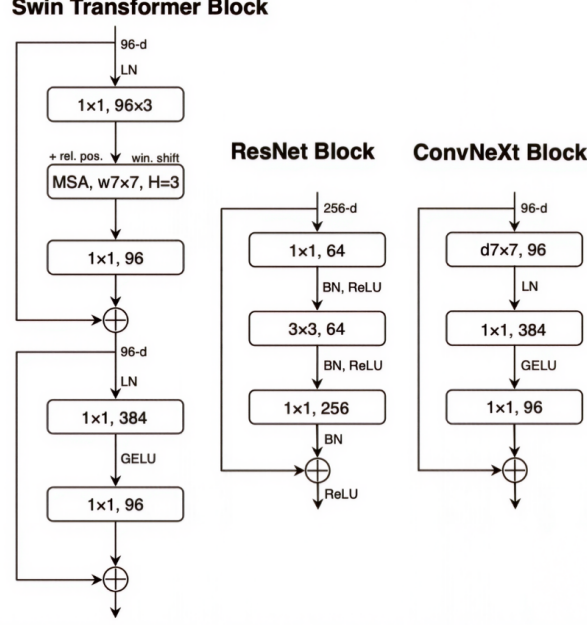


Figure 3.1: *ConvNeXT model architecture*

of BatchNorm, GELU activations, and simplified block wiring that separates spatial (depthwise) and channel (pointwise) mixing. These changes yield competitive accuracy and scalability across standard vision benchmarks. Initialization is performed from the public facebook/convnext-base-224-22k checkpoint, and the model is adapted to the task via supervised fine-tuning. The classification head is configured to produce logits over the project’s class set [11].

Although ConvNeXt is an image backbone, the inputs are time–frequency ‘images’: single-channel log-Mel spectrograms with 128 Mel bands and a time frame number that varies. Here the mono-spectrogram tensors of shape $[B \times 1 \times 128 \times 497]$ are utilized, with front-end parameters (STFT with $n_fft = M = 1024$, hop length = 320; power spectrogram; Mel scale with $H = 128$ and $STFT = 513$; dB conversion with $top_{db}=80$; normalization with mean -4.268 and std 4.569) to be compatible between preprocessing and expected input statistics of this backbone. In line with the matching spectrogram modality, the input channel number of that backbone is set to 1 (mono-channel), with its pre-trained spatial weights retained and its final classification head replaced with that of appropriate output dimensionality. [11].

The model operates in a fully convolutional manner up to a global pooling stage, after which a linear classification head maps to the species label space. At training time, the model’s logits are treated as class scores, and task-specific loss is computed. During evaluation, the logits are converted to normalized class probabilities using a *softmax* function before computing metrics. This deployment follows from the recommended image-classification configuration of ConvNeXT in its original description and that of the Hugging Face model cards [27, 11].

Choosing ConvNeXT provides (i) strong inductive biases for local pattern learning that suit spectrogram ‘textures’, (ii) compatibility with large-scale ImageNet-22k pretraining for improved

transfer, and (iii) an efficient hierarchical design whose patchify stem and depthwise convolutions scale well to long time axes typical of audio spectrograms [27, 11]. The BirdSet variant confirms this practice by publishing a ConvNeXT-Base checkpoint explicitly trained on 5 s bird vocalization clips using single-channel spectrogram inputs, which follows in the pipeline [27].

3.5 ECA-Net

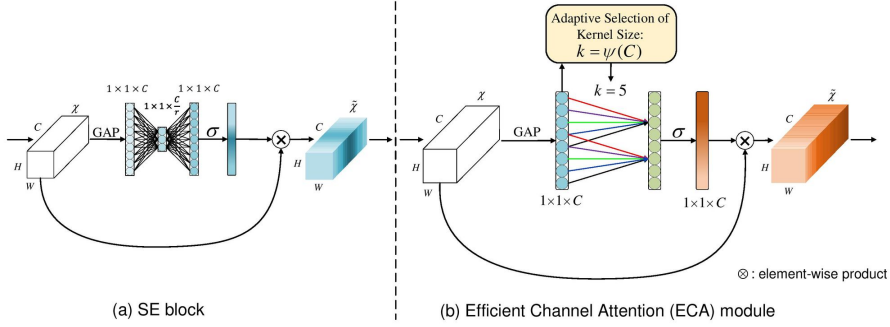


Figure 3.2: *ECA module overview (adapted from [53]; see also [35]).*

The ECA-Net system implements a channel-attention block which operates on convolutional backbones without reducing their dimensionality. The first operation of the module applies global average pooling to obtain one descriptor per channel from the convolutional feature map. ECA-Net replaces the traditional fully connected bottleneck of SE blocks by using a 1- D convolution to process channel descriptors for local cross-channel interaction detection. The sigmoid gate transforms these responses into channel weights which then scale the original feature map through multiplication. The 1- D convolution kernel size adjusts automatically based on channel numbers to ensure the attention receptive field expands properly with increasing model width. The depth-wise operation maintains minimal added cost while remaining compatible with normalizer-free backbones including NFNet. The ECA-NFNet model operates as the feature extractor while its classification head employs sigmoid activation to generate independent probabilities for each species which work for one-vs-rest evaluation. The ECA block schematic appears in figure 3.2 [53, 35].

3.6 Focal Loss

In BirdCLEF, class frequencies are highly skewed: some species occur often, while many appear rarely. After segmenting recordings into non-overlapping 5-second clips, most segments contain *no vocalization* from a given species, so training presents a large majority of ‘easy negatives’. Moreover, because segmentation is blind to the exact vocalization timing, some positive recordings yield few (or zero) truly positive segments, introducing additional sparsity

and label noise at the segment level. Under standard cross-entropy, gradients are dominated by these abundant, easy negatives, which can hamper learning for rare species. Focal loss addresses this by down-weighting well-classified examples and emphasizing hard (misclassified) examples. Definition: Let $z \in \mathbb{R}$ be the logit, $p = \sigma(z)$ the predicted probability, and $y \in \{0, 1\}$ the label, then,

$$p_t = \begin{cases} p & , \quad y = 1 \\ 1 - p, & y = 0 \end{cases}$$

The softmax probabilities denote p_t as the predicted probability of the true class. The focal loss for this sample is

$$\text{FL}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t + \varepsilon),$$

where an easy example contribution (large p_t) is attenuated through focusing parameter $\gamma \geq 0$, $\alpha_t \in [0, 1]$ optionally re-weights classes, and ε prevents $\log 0$. In these experiments, $\gamma = 2, \alpha_t = 1$ is set, meaning no explicit class re-weighting, relying on the modulation factor $(1 - p_y)^\gamma$ to diminish loss from easy background-like segments and common species while preserving gradients on hard or rare cases. The training objective averages this loss over a mini-batch:

$$\mathcal{L} = \frac{1}{B} \sum_{i=1}^B \left[-\alpha_{y_i} (1 - p_{y_i}^{(i)})^\gamma \log(p_{y_i}^{(i)} + \varepsilon) \right]$$

Because segments are extracted without aligning to precise call onsets, many ‘positive’ recordings yield few truly positive 5s windows and numerous background windows. Focal loss suppresses the overwhelming mass of easy negatives and easy positives alike, thereby allocating more learning signal to the comparatively scarce hard segments where species cues are faint, brief or partially captured, particularly benefiting rare species [26].

3.7 AUC-ROC

The distribution of species in BirdCLEF is extremely skewed. AUC-ROC is indifferent to any individual operating threshold and consolidates the ranking quality across all thresholds, making it appropriate for highly imbalanced recognition and complementary to focal loss. For a given species c , its classification is addressed in a one-vs-rest setting. Let $(s_i^{(c)}, y_i^{(c)})_{i=1}^N$ denote the predicted scores $s_i^{(c)} \in [0, 1]$ and the binary labels $y_i^{(c)} \in \{0, 1\}$. Sweeping a threshold τ produces

$$\text{TPR}(\tau) = \frac{\text{TP}(\tau)}{\text{P}(\tau)}, \quad \text{FPR}(\tau) = \frac{\text{FP}(\tau)}{\text{N}(\tau)}$$

The ROC curve plots $\text{TPR}(\tau)$ vs. $\text{FPR}(\tau)$ as τ varies from 1 to 0. The per-species AUC-ROC is the area under this curve,

$$\text{AUC} = \int_0^1 \text{TPR} d(\text{FPR}),$$

Equivalently, the probability that a randomly chosen positive example receives a higher score than a randomly chosen negative example [32]:

$$\text{AUC} = \frac{1}{N_+ N_-} \sum_{i: y_i^{(c)}=1} \sum_{j: y_j^{(c)}=0} \left[\mathbf{1}\{s_i^{(c)} > s_j^{(c)}\} + \frac{1}{2} \mathbf{1}\{s_i^{(c)} = s_j^{(c)}\} \right]$$

Where, $N_+^{(c)} = \sum_{i=1}^N y_i^{(c)}$ is the number of positives for class c ; $N_-^{(c)} = N - N_+^{(c)}$ is the number of negatives [12].

Macro averaging with class filtering: in the evaluation split, some species do not have any positive instances. As AUC is undefined without one positive and one negative, those species are removed from the scoring. Let $\mathcal{C}_+ = \{c : N_+^{(c)} > 0\}$, Then the reported metric is *macro average* across the remaining species:

$$\text{Macro AUC} = \frac{1}{|\mathcal{C}_+|} \sum_{c \in \mathcal{C}_+} \text{AUC}$$

Macro averaging places equal emphasis on all scored species, damping down dominance from frequent classes and more accurately reflecting performance on rare species [32, 54].

In the project setting, scoring ignores identifiers (row IDs) but uses numeric prediction columns that correspond to species. Predictions must be numeric; non-numeric values render a submission invalid. Class-wise AUC-ROC values are calculated on a one-vs-rest basis and then macro-averaged across species that have at least one positive instance present in ground truth for the evaluated fold/split. This follows standard leaderboard evaluation procedures for multi-class ecological sound recognition, which ensures robustness to class balance and threshold selection.

3.8 Adam

The model parameters are optimized using the Adam, a stochastic gradient-based method that combines the benefits of momentum and adaptive learning rates. Adam maintains exponentially decaying estimates of the first moment (mean) and the second moment (uncentered variance) of the gradients and uses these estimates to adapt the learning rate for each parameter individually [21]. Let g_t denote the gradient loss with respect to the iteration parameters t . The first-moment

and second-moment estimates are updated as

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Where m_t is the estimate of the first moment, v_t is the estimate of the second moment, and $\beta_1, \beta_2 \in [0, 1)$ are exponential decay rates controlling the weighting of past gradients and squared gradients, respectively. Because these moment estimates are biased towards zero during the initial steps, bias-corrected estimates are computed as

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

The parameter update rule is then performed as follows:

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}, \quad (3.1)$$

where α is the learning rate and ε is a small constant for numerical stability.

The project uses Adam as an optimizer with 5×10^{-5} learning rate and $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\varepsilon = 10^{-8}$ (default values from [21]) to achieve stable and efficient convergence for deep neural networks. The optimizer’s ability to adaptively scale the learning rates per parameter is particularly beneficial for training large models such as ConvNeXT, where different layers may require different step sizes during fine-tuning.

3.8.1 Learning Rate (α)

In Adam, the learning rate α serves as a universal step size that modifies all parameter updates. The update process in 3.1 applies a practical step of α divided by the local gradient variability, measured through $\sqrt{\hat{v}_t}$, to each parameter. Adam divides the learning rate step by the square root of each parameter’s gradient variability to perform individual adjustments. This study maintains a static learning rate value of $\alpha = 5 \times 10^{-5}$ without implementing any learning rate scheduling approach. This work uses this conservative learning rate for stable fine-tuning of the large pre-trained ConvNeXT backbone on log-Mel spectrograms. The conservative learning rate prevents sudden changes in early convolutional blocks and reduces the likelihood of catastrophic forgetting in early layers. A learning rate between 10^{-4} and 10^{-5} is suitable for full model adaptation of robust backbones, but optimization performance can be enhanced through small adjustments between these two values.

3.9 Softmax

For the ConvNeXT model, the final classification layer produces an unnormalized score vector $\mathbf{z} \in \mathbb{R}^C$ that contains C species classes. The scores known as logits undergo transformation into normalized class probabilities through the softmax function [2]. The softmax probability for class c among 1 to C is defined as

$$p_c = \frac{\exp(z_c)}{\sum_{j=1}^C \exp(z_j)}$$

This transformation ensures that $p_c \in (0, 1) \forall c$, and $\sum_{c=1}^C p_c = 1$. The softmax function has two essential features as follows: **Normalization**, the softmax function transforms any real-valued logits into a valid probability distribution across the C classes. **Relative scaling**, the probability assigned to a class depends not only on its own logit but also on the logits of all other classes, capturing relative confidence.

Softmax outputs serve as input for loss functions during training to measure the difference between predicted and actual class labels. During testing, the model produces probabilities which enable the computation of ranking-based metrics such as per-class AUC-ROC by treating p_c as the confidence score of class c in one-vs-rest scenarios. The softmax activation function in the output layer generates probability distributions across the 206 bird species while maintaining predictions that sum to one and directly represent class membership probabilities.

3.10 F1, Precision and Recall

The performance evaluation for single-label multi-class prediction uses one-vs-rest methods for each species $c \in \{1, \dots, 206\}$. The per-class confusion matrix contains four standard entries which are TP, FP, FN and TN, while $S = TP + FN$ represents the class support. Precision, recall, and F1 are

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{F1} = \frac{2 \text{ Precision Recall}}{\text{Precision} + \text{Recall}}$$

The harmonic mean of precision and recall is defined as the F1 metric according to [29, 42, 43]. The standard practice in machine learning reporting defines zero as the value of undefined metrics when their denominators become zero (for example no predicted positives).

Two complementary averages are reported because species frequencies are imbalanced:

$$\text{Macro-Metric} = \frac{1}{C} \sum_{c=1}^C \text{Metric}_c \tag{3.2}$$

The method uses equal weights for all species in the calculation. The weighted sum of a metric across all classes is calculated as

$$\text{Weighted-Metric} = \sum_{c=1}^C \frac{S_c}{\sum_{j=1}^C S_j} \text{Metric}_c \quad (3.3)$$

Where C = total number of classes (206). Metric_c per-class precision, recall, or F1-score for class and S_c = support, that is, the number of validation samples belonging to the class. The method uses observed class frequencies to determine the weights for each species. Macro averages are sensitive to rare species, while weighted averages reflect performance on the observed distribution [29].

3.11 Sigmoid

The ECA-NFNet branch produces separate species probabilities by using the logistic sigmoid activation function. The probability for each class c in time window t is calculated from logits $z_{t,c} \in \mathbb{R}$ as follows:

$$p_{t,c} = \sigma(z_{t,c}) = \frac{1}{1 + \exp(-z_{t,c})}$$

The output functions as an independent Bernoulli likelihood in the multi-label framework. The sigmoid activation function does not enforce the constraint $\sum_c p_{t,c} = 1$ which allows the probabilities of multiple species to be high simultaneously, which makes it suitable for overlapping vocalizations and class-based evaluation (for example AUC-ROC) [46]. The decision-making process involves applying a per-class threshold $\tau_c \in [0, 1]$ to the output:

$$\hat{y}_{t,c} = \mathbb{I}\{p_{t,c} \geq \tau_c\}$$

The species-specific threshold τ_c can be optimized on the validation data to achieve a balance between precision and recall for each species. Standard calibration techniques such as sigmoid calibration can be applied to held-out data to obtain calibrated probabilities without modifying the architecture [47, 42]. The ECA-NFNet backbone serves as the feature extractor in this implementation, while the sigmoid layer defines the classification head output activation [3, 53].

3.12 K-fold cross validation

To achieve unbiased generalization performance estimates and model selection adjustments independent of the public leaderboard the k -fold cross-validation procedure with stratification is implemented for the ECA-NFNet model. This set of restrictions stops the model from accessing

segments from the same recording. Let the metadata table be

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N,$$

where x_i is a 5 s log-Mel spectrogram and $y_i \in \{1, \dots, 206\}$ is the primary species label. K disjoint index sets $\{\mathcal{I}_1, \dots, \mathcal{I}_K\}$ such that [17]

$$\bigcup_{k=1}^K \mathcal{I}_k = \{1, \dots, N\}, \quad \mathcal{I}_k \cap \mathcal{I}_\ell = \emptyset \ (\forall k \neq \ell),$$

with stratification by class: for each class c , the per-fold counts satisfy

$$\left| \{i \in \mathcal{I}_k : y_i = c\} \right| \approx \frac{N_c}{K}, \quad N_c := \left| \{i : y_i = c\} \right|.$$

The data shuffle method minimizes variance in addition to producing results that can be reproduced because a fixed random seed is used for splitting [4].

For fold $k \in \{1, \dots, 5\}$, here it is defined as,

$$\mathcal{D}_{\text{train}}^{(k)} = \bigcup_{\ell \neq k} \{(x_i, y_i) : i \in \mathcal{I}_\ell\}, \quad \mathcal{D}_{\text{val}}^{(k)} = \{(x_i, y_i) : i \in \mathcal{I}_k\},$$

train the model on $\mathcal{D}_{\text{train}}^{(k)}$, and evaluate on $\mathcal{D}_{\text{val}}^{(k)}$.

Per-fold protocol and model selection: the data order and weight initialization becomes deterministic before each fold k begins. The validation data $M_{\text{val}}^{(k)}(t)$ receives statistical computation at each training iteration point after each epoch. The maximum validation metric AUC-ROC is used to calculate the best $M_{\text{val}}^{(k)}(t)$ for fold k after each epoch t . The best per-fold score is $M^{(k)} = \max_t M_{\text{val}}^{(k)}(t)$ the corresponding model parameters are saved using the torch save function in PyTorch Tensor (.pt) extension, and further the retained final epoch weights are used for inference.

Aggregation across folds: the best validation score from each fold is denoted by $M^{(k)}$. According to [52, 25] the cross-validated estimate and dispersion follow this structure.

$$\widehat{M} = \frac{1}{K} \sum_{k=1}^K M^{(k)}, \quad s^2 = \frac{1}{K-1} \sum_{k=1}^K (M^{(k)} - \widehat{M})^2$$

The standard error of the estimate is given by $\text{SE} = s/\sqrt{K}$ and the $(1 - \alpha)$ confidence interval can be written as

$$\widehat{M} \pm t_{K-1, 1-\alpha/2} \frac{s}{\sqrt{K}}$$

The main label helps stratification to preserve class distribution between folds while the training

batch order is randomized but validation batch order remains fixed. The fixed random seed enables the reproduction of stratified divisions and training paths k between runs [17]. On the other hand, for the ConvNeXT model, the k -fold cross-validation was not applied because the training process was very time-consuming and limited in computational resources. Instead, a fixed validation split was used to monitor performance.

3.13 Data Augmentation

The ECA-NFNet model receives five different augmentations during training to enhance its ability to generalize under class imbalance and background variability conditions [39].

Time/Frequency masking: The log-Mel spectrogram contains M Mel bands and T frames, which form the matrix S [39]. SpecAugment masks contiguous frequency bands and time frames as

$$S[m, t] \leftarrow 0 \quad \text{for } m \in \mathcal{M}_f \subset \{1, \dots, M\}, t \in \mathcal{M}_t \subset \{1, \dots, T\}$$

Time/tempo jitter: The waveform $x(t)$ undergoes resampling through the rate factor $s > 0$ which generates $x_s(t) = x(st)$ (for example $s \in \{0.9, 1.0, 1.1\}$) to create minor time-scale modifications that maintain label meaning and introduce temporal pattern variations [23]

Random gain: The amplitude of waveforms undergoes random scaling through a positive factor $g > 0$ which produces $x'(t) = g x(t)$ to model loudness variations caused by microphone distance and device capture [6, 34].

Background soundscape mixing: The addition of background segments (music/speech/noise) to $x(t)$ at controlled SNR produces $x'(t) = x(t) + \alpha n(t)$ where n originates from a curated noise corpus [48].

MixUp/CutMix. The MixUp method generates new training examples by combining two input pairs (x_i, y_i) and (x_j, y_j) through $\tilde{x} = \lambda x_i + (1 - \lambda)x_j$ and $\tilde{y} = \lambda y_i + (1 - \lambda)y_j$ where λ ranges from 0 to 1. The method of decision boundary regularization through MixUp produces better calibration results [57]. CutMix replaces a random time-frequency patch of one spectrogram with that of another and mixes labels by patch area, encouraging attention to distributed evidence [56]. The practice of mixing waveforms and spectrograms between different classes has been widely used in sound recognition tasks [50].

4 Model Evaluation

4.1 ConvNeXt Model

The 5 s segment dataset serves as the evaluation space to assess the performance of the model following the preparation approach detailed in (3). The ConvNeXT model measures two quantities at each end point of the epoch: (i) the average focal loss of the training phase (3.6) and (ii) class-wise scores converted from logits via softmax (3.9). The system focuses primarily on threshold-free classification, such as AUC-ROC (3.7), and from another perspective, metrics such as precision, recall and F1-score (3.10). The system saves checkpoints after each epoch, and the results presented for each run correspond to the checkpoint from the last epoch.

The ConvNeXT model is trained for a fixed number of 5 epochs, after which the final checkpoint is saved. Choosing the appropriate number of epochs involves balancing sufficient learning of task-specific characteristics against the risk of overfitting. A small number of epochs is particularly chosen when fine-tuning a large pretrained model.

A similar strategy is frequently used in the transfer learning literature. For example, in fine-tuning large models on downstream tasks, researchers often report using between 3 and 10 epochs when starting from strong pretrained weights (few enough to avoid overfitting), but sufficient to adapt to the target distribution [8, 28]. The 5-second segmented audio clip dataset size and ConvNeXT high capacity make training for five epochs a practical solution to expose bird call patterns adequately without overtraining redundant or noisy segments.

4.1.1 AUC-ROC Diagnostic

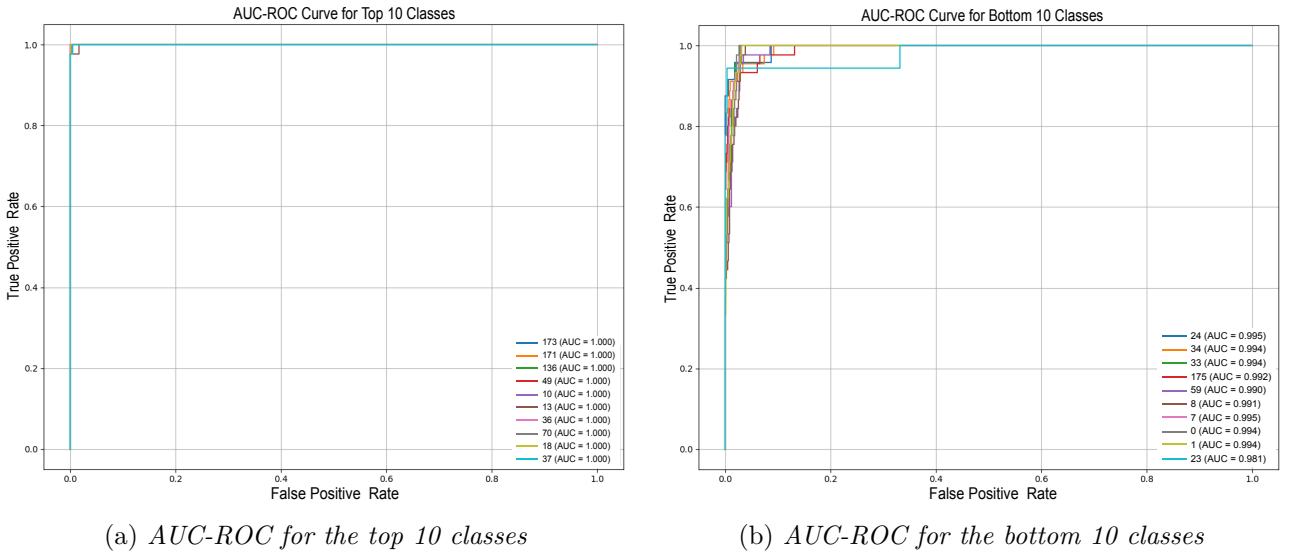


Figure 4.1: AUC-ROC for bird species

The per-class AUC-ROC analysis shows virtually flawless separability for the strongest classes and very high separability even for the weakest classes. The top-10 classes (figure 4.1a) are indistinguishable from 1.000 AUC-ROCs with curves closely following the left and top borders; high true-positive rates are achieved at virtually zero false-positive rate. The bottom-10 classes (figure 4.1b) still show AUCs within the 0.981–0.995 range. The ROC curves show a slight axis shift at low FPR values due to the model output of a small number of initial false positive outputs before reaching its optimal sensitivity.

The classifier shows reliable performance in ranking species with different threshold values. The remaining gap for the weakest classes probably stems from class imbalance or overlapping acoustic signatures or segment-level noise from non-overlapping 5 s windows. Macro-AUC is dominated by strong performance, while per-class operating thresholds should be calibrated for the few lower-AUC-ROC species, targeted augmentation or additional labeled audio is expected to yield the most significant gains.

4.1.2 Precision-Recall Diagnostic

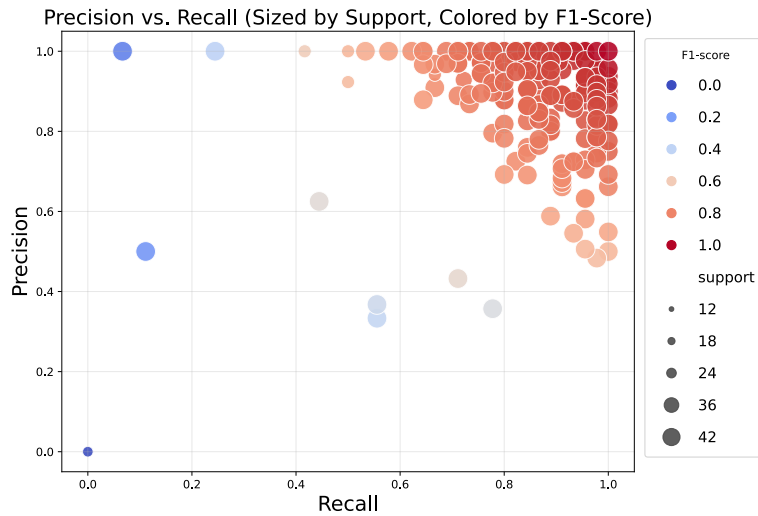


Figure 4.2: *Precision vs. recall per class; marker size $\propto$ support, color $\propto$ F1-score.*

The precision–recall scatter plot in the figure 4.2 represents performance by species through markers that display each class with recall values on the x-axis and precision values on the y-axis. The marker size reflects support and the color represents the F1-score, which computes the harmonic mean between precision and recall. The majority of classes in the upper-right section of the plot have precision and recall values higher than 0.9 and 0.8, respectively, with dark red coloring, indicating high precision and recall values. The upper-left region shows outliers with high precision but reduced recall, while the lower-right region shows outliers with reduced precision at high recall levels. The smallest markers (low-support species) tend to have lighter colors and more dispersion, reflecting the impact of data scarcity on F1. Most species

perform strongly according to the pattern, but reveal two primary error sources, classes with low support and confusable classes require more data or additional augmentation.

4.1.3 F1-score Diagnostics

The eCDF of the F1-scores per-class appears in the figure 4.3. The F1-score values appear on the horizontal axis, while the proportion of species with F1 scores at or below that value appears on the vertical axis. The curve shows a substantial shift to the right and stays near zero until it reaches the mid-0.7 range before rapidly increasing between 0.8 and 0.95, indicating that most species reach high F1 scores. The low-F1 tail (left side) contains a small minority of species, which means classes with scarce support, overlapping vocalizations, or noisy segments. The curve includes selected class identifiers that mark tail cases for error analysis and data augmentation. eCDF provides a condensed view of class performance without binning, which allows immediate identification of quantiles (for example, the median falls within the high-F1 range) while supplementing the precision–recall scatter plot 4.2.

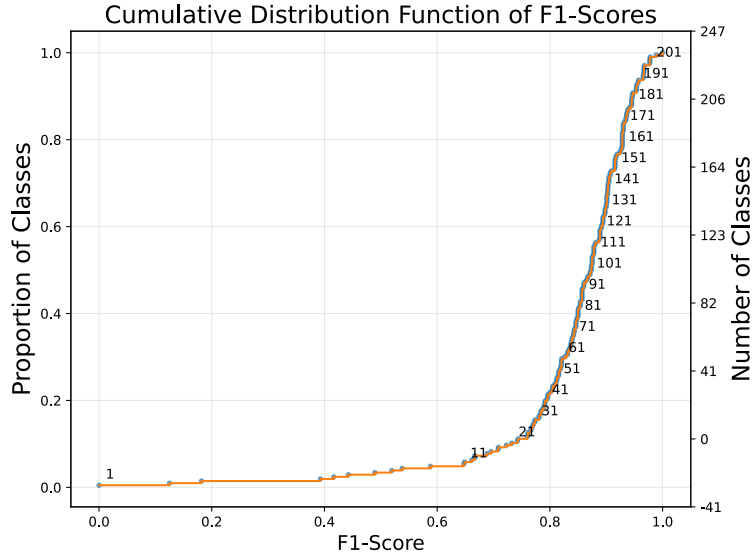


Figure 4.3: *eCDF of per-class F1-scores (206 species). Points are shown for every 10th class*

The evaluation takes place in *validation split* (20% of the segmented dataset), which comprises $N_{test} = 9,114$ five-second segments. The model achieves an accuracy of 0.846. The macro-averaged scores (each species weighted equally) are: precision 0.874, recall 0.840, and F1-score 0.842. The scores, weighted by class support, are: precision 0.875, recall 0.846, and F1-score 0.846. The macro and weighted F1-score are 0.842 and 0.846 respectively, with a gap of 0.004, indicating broadly consistent class-wise performance, with slightly higher effectiveness on more frequent species. The higher macro precision compared to recall shows a slightly conservative approach, with fewer false positives but some missed cases. This matches the use of non-overlapping 5-second segments and the presence of inter-species acoustic overlap.

4.2 ECA-Net Model (External Training)

In this project, pre-trained weights that were fine-tuned specifically on the BirdCLEF 2025 dataset of bird vocalizations are used. The backbone is ECA-NFNet, which augments a CNN with a lightweight channel-attention module that *avoids dimensionality reduction* and captures *local cross-channel interactions* through a 1D convolution whose kernel size is adaptively chosen as a function of the channel dimension [53]. This design yields accuracy gains with negligible parameter/FLOP overhead, making it suitable for spectrogram classification.

The model was trained using a standardized pipeline, as described in [37]. To construct examples, the audio is cropped into fixed-length segments using a *crop-audio* procedure. The class imbalance is mitigated by *oversampling species with very few samples* (metadata files provided for < 20 examples). Heavy-augmentation with a number of epochs 10 and 5-fold was used. Training can proceed in *multiple stages* with the pause and resume of the checkpoints, and this are validated with *per-class AUC-ROC* utilities tailored to the competition format [38]. The ECA-NFNet weights used here were not trained in this workflow, only the *frozen external weights* are used for inference within this project pipeline; moreover, no additional fine-tuning is performed. All metrics reported here are computed on a validation split to ensure comparability with the ConvNeXT-based model.

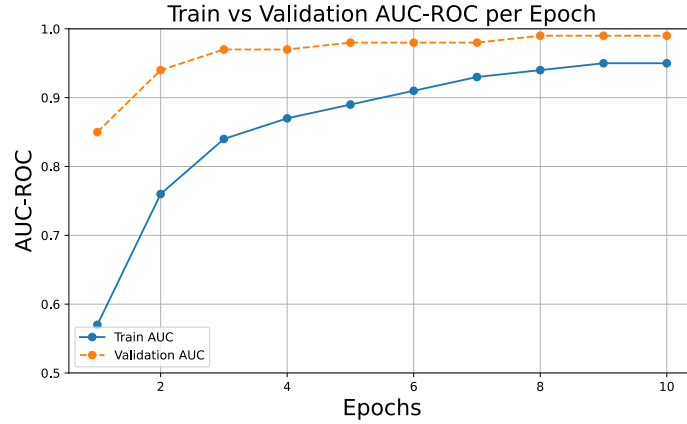


Figure 4.4: *Train vs. validation AUC-ROC per epoch for the ECA-Net model*

Figure 4.4 plots train vs. validation AUC-ROC per epoch. The validation performance increases steeply from ≈ 0.85 at epoch 1 to ≈ 0.96 by epoch 3 and plateaus near 0.99 from epochs 8–10. Training AUC increases more gradually (from ≈ 0.57 to ≈ 0.95), with no late-epoch collapse. The persistent gap $\text{AUC}_{\text{val}} > \text{AUC}_{\text{train}}$ (about 0.03–0.04 at convergence) indicates strong regularization or effectively noisier training data such as on-the-fly augmentation [18], class rebalancing or segment-level label sparsity, rather than overfitting. Since the validation AUC saturates by $\sim$ epoch 8, early stopping at this point is justified; further epochs yield negligible gains.

5 Inference: Weighted Blend

This project combines two heterogeneous CNN backbones at inference time using a weighted average of their predicted class probabilities. Ensembling models with complementary inductive biases is a standard way to reduce variance and improve robustness [24]. The backbones are *ConvNeXT* and *ECA-NFNet* [3, 27].

ConvNeXT inference: Inference mirrors the training pipeline as each waveform is resampled to 32 kHz, converted to mono by channel averaging, and then transformed into a magnitude spectrogram with parameters(`n_stft=513`, `n_fft=1024`, `hop_length=320`, `power=2.0`). The input for the ConvNeXT input stays the same (refer to Section [2.2.2,3.1]). The resulting log-mel tensor is fed to a ConvNeXT image classification backbone with a linear head over $C = 206$ species, using a fine-tuned checkpoint loaded before evaluation. Logits $\mathbf{z} \in \mathbb{R}^C$ are produced, and the class probabilities are obtained by applying a softmax over the class dimension. This ConvNeXT branch produces a C -dimensional probability vector for each input segment, which is stored in `submission0.csv`.

ECA-NFNet inference: For each soundscape file, the waveform is split into non-overlapping windows of length 5 s, converted to log mel-spectrograms, and passed through the ECA-NFNet encoder with an attention head. The model returns chunk-wise logits $\mathbf{z} \in \mathbb{R}^{T \times C}$ ($T=12$ chunks, $C=206$ classes), which are mapped to probabilities by a per-class `sigmoid` 3.11.

(1) *Post-processing with power adjustment for low rank:* Let $M_c = \max_t \mathbf{p}_{t,c}$ and let π be a permutation sorting classes by $M_{\pi(1)} \geq \dots \geq M_{\pi(C)}$. Define the tail set $\mathcal{T} = \{\pi(k) : k > \text{top_k}\}$ with `top_k=30`. For all $c \in \mathcal{T}$, apply a class-wise power transform with exponent $\alpha = 2$:

$$\mathbf{p}_{t,c} = \left(\mathbf{p}_{t,c}\right)^\alpha \quad \forall \alpha = 2, \quad c \in \mathcal{T}$$

Leaving the top- k classes unchanged. Since $\mathbf{p}_{t,c} \in [0, 1]$, $\alpha > 1$ attenuates low-probability activations for low-ranked classes while preserving the head classes.

(2) *Temporal smoothing across chunks:* In this project, a simple 3-point moving average is applied to each class column with kernel `[0.2, 0.6, 0.2]`: $\tilde{p}_{t,c} = 0.2 \bar{p}_{t-1,c} + 0.6 \bar{p}_{t,c} + 0.2 \bar{p}_{t+1,c}$. In the first and last frames, nearest-neighbor padding is used ($\bar{p}_{0,c} = \bar{p}_{1,c}$, $\bar{p}_{T+1,c} = \bar{p}_{T,c}$), which reduces to $\tilde{p}_{1,c} = 0.9 \bar{p}_{1,c} + 0.1 \bar{p}_{2,c}$ and $\tilde{p}_{T,c} = 0.9 \bar{p}_{T,c} + 0.1 \bar{p}_{T-1,c}$. The smoothed matrix $\tilde{\mathbf{p}} \in \mathbb{R}^{T \times C}$ is then used for submission. The resulting $\tilde{\mathbf{p}}$ (shape $T \times C$) is written as per-chunk rows for the `submission1.csv` table and used in the weighted downstream blend. The ECA-NFNet branch used in this project was not run for inference as part of this work; therefore, only a brief high-level description is included here. For more inference details, see the original [37, 38].

5.1 Weighted Blending

The predictions from two independent classifiers are combined through soft voting (weighted probabilistic averaging) at test time according to [45, 5]. Let r index a submission row (`row_id`) and $c \in \{1, \dots, C\}$ index species. Denote by $p_{r,c}^{(0)}$ and $p_{r,c}^{(1)}$ the probabilities per species produced by the two models for row r . Given non-negative blend weights (w_0, w_1) with $w_0 + w_1 = 1$, the blended score for species c in row r is the convex combination

$$\hat{p}_{r,c} = w_0 p_{r,c}^{(0)} + w_1 p_{r,c}^{(1)}, \quad (w_0, w_1) = (0.75, 0.25) \quad w_0, w_1 \geq 0$$

The operation is applied element-wise to all species columns after aligning the two prediction tables by `row_id`. The blended values satisfy $\hat{p}_{r,c}^k \in [0, 1]$ (where if $k = 0$ means the output of the ConvNeXT model and $k = 1$ means the output of the ECA-NFNet model) and preserve the classification structure beneficial for threshold-free metrics such as AUC-ROC. Soft voting is a standard ensembling strategy when base learners output probabilities, often improving robustness by aggregating complementary errors across models [45, 5].

The development process lacked access to ground-truth labels for the test split so cross-validated tuning of blend weights became impossible. Instead, a small set of candidate convex combinations was tried, and the pair that yielded the best score on the hidden holdout was retained; no exhaustive search was performed. This is a pragmatic use of fixed-weight *soft voting* (probability averaging) when base learners output class probabilities [45]. A known caveat is that repeated adaptation to feedback from a fixed holdout can bias estimates (*adaptive overfitting*); consequently, only a few trials were attempted, and the final weights are reported for transparency [9].

The final submission matrix is formed by retaining `row_id` and replacing each species column with its blended score $\hat{p}_{r,c}$; the result is written for evaluation. This probability-level blending is model-agnostic and requires only consistent species column naming and exact row alignment between the two prediction tables. Blending assumes comparable, well-formed probabilities over the same label space. When mixing `softmax` (multi-class) and `sigmoid` (multi-label) heads, ensure the target convention and evaluation protocol are consistent before combining. The final predictions for each of the 206 species are independent and can have a sum of more than 1, as each predicted probability is independent from the others. Refer to table (5.1).

Table 5.1: *Example for final predictions CSV file for 5 s chunk*

row_id	1139490	1192948	...	126247	yeofly1
H02_190500_5					
(prediction value)	0.233493502	0.00345928094	...	0.00395615046	0.00581944248

6 Conclusion

The priority involved creating a reliable bird species ranker that needed to operate under extreme conditions, including a limited 90-minute inference window, along with no Internet and no GPU acceleration. The resulting pipeline maintains a purposeful design for straightforward operation along with complete audibility. The system transforms audio signals into normalized log-mel features that proceed through ConvNeXT with a softmax head that produces single probability vectors and ECA-NFNet with heavy-augmentation, a sigmoid head and light temporal smoothing that produces window-wise probabilities. The system combines these outputs by performing a fixed weighted blend of their values after strict alignment on `row_id` across each column. Focusing on simplicity instead of complexity allows for reliable run-time performance and reproducible results when working in CPU-only offline environments [22].

The system achieved outstanding ranking results despite operating under its defined constraints by achieving **0.880** AUC-ROC on the 30% hidden subset and **0.861** AUC-ROC on the complete hidden dataset. The ensemble maintains its ordering capability when moving from a small data split to the full evaluation dataset while showing slight overfitting because the performance difference amounts to $\Delta \approx 0.019$. The combination of two separate heads that fuse at the probability level achieves robust and transferable performance when faced with limited computational resources and time constraints.

The project provides a practical solution for deploying systems under constraints through its approach, which includes deterministic preprocessing, lightweight inference, and an easy-to-maintain transparent blend system. A well-engineered system with diverse model heads produces reliable AUC-ROC results when working within restricted operational boundaries, serving as a basis for subsequent performance enhancements.

6.1 Limitations

The system operated under specific operational constraints, including a 90-minute inference period, no Internet access, and no GPU processing. The simplicity of this pipeline and the audibility of the constraints provided restrictions on the investigation of advanced decoding methods, augmentation testing, and stacking techniques (stacking more models). The selection of blend weights used heuristic methods instead of validating on out-of-fold data, and ensemble processing occurred without probability calibration, which might affect the comparison between the outputs of softmax ConvNeXT and sigmoid ECA-NFNet [40, 28].

The external implementation of ECA-NFNet exists only for inference and post-processing activities because this project did not train the model. ECA-NFNet model is trained at a higher level whereas this was not possible for the model ConvNeXT training due to limited CPU and

GPU source which is required for large audio data. The use of numeric `primary_id` instead of eBird codes resulted in species identification failures, which blocked external data from being merged for input enhancement and prior knowledge access. The hidden data (labels unavailable) prevented hyperparameter tuning, which limits the optimization of blend weights or thresholds to fixed selections only [58, 53]. The STFT/mel configuration was the only feature extraction method; the evaluation process omitted other window, hop size combinations and mel scale variations which is required for different sizes of mel-spectrograms.

6.2 Potential Improvements

The existing project requires specific extensions to achieve better performance reliability without introducing unacceptable latency. The pseudo-labeling method allows the system to produce labels for unlabeled or weekly unlabeled audio data and filter them through confidence thresholds before adding them to the training process. The system then can detect unusual species through its self-training process without needing external data. The ConvNeXT model trained in this project can also be enhanced by the introduction of heavy-augmentation techniques, including SpecAugment, time/tempo jitter, random gain, background soundscape mixing, and mixup/cutmix, and not only for the ECA-NFNet model training. This will enable the training distribution to expand and will enhance both generalization performance and noise robustness. [39].

In order to improve score, stacking more faster and lightweight models together and trained on subsets of data will analyze better because this method will separate the handling of frequent and rare species. The models need to use combined probability levels to achieve balanced results (for example same logit functions). The capability to export trained weights into ONNX files for running ONNX runtime will help in faster inference. The hardware-independent inference framework works offline with CPUs only without requiring GPU support [1, 33].

Moreover, learning blend weights through grid search learning while performing probability calibration with temperature or isotonic scaling techniques, will incorporate optimal values for parameters. After identifying eBird codes, the use of bird detection models (such as Perch or BirdNet) to detect bird vocals from raw audio recordings can help in bird sound separation from background noise. Therefore, the pipeline will achieve better accuracy with robustness and predictable latency by implementing these extensions [13, 7, 15].

References

- [1] Bradley C. Boehmke and Brandon M. Greenwell. *Hands-On Machine Learning with R*. Discussion of stacking ensembles, grid search for blending, and systematic model comparison. 2020. URL: <https://bradleyboehmke.github.io/HOML/stacking.html> (cited on p. 26).
- [2] J. S. Bridle. “Probabilistic Interpretation of Feedforward Classification Network Outputs, with Relationships to Statistical Pattern Recognition”. In: *Neurocomputing*. Ed. by F. F. Soulié and J. Héroult. Vol. 68. NATO ASI Series. Springer, Berlin, Heidelberg, 1990, pp. 227–236. DOI: 10.1007/978-3-642-76153-9_28. URL: https://link.springer.com/chapter/10.1007/978-3-642-76153-9_28 (cited on p. 15).
- [3] Andrew Brock et al. “High-Performance Large-Scale Image Recognition Without Normalization”. In: *arXiv preprint arXiv:2102.06171* (2021). Accessed 2025-08-21. URL: <https://ar5iv.labs.arxiv.org/html/2102.06171> (cited on pp. 16, 23).
- [4] scikit-learn developers. *StratifiedKFold*. Stratified k-fold splits preserving class proportions. 2023. URL: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedKFold.html (cited on p. 17).
- [5] Thomas G. Dietterich. “Ensemble Methods in Machine Learning”. In: *Multiple Classifier Systems*. Springer, 2000, pp. 1–15. URL: <https://web.engr.oregonstate.edu/~tgdp/publications/mcs-ensembles.pdf> (cited on p. 24).
- [6] DigitalOcean Community. *Data Augmentation for Audio: Noise Injection, Time Stretch, Pitch Shift, Volume Perturbation*. Aug. 18, 2025. URL: <https://www.digitalocean.com/community/tutorials/data-augmentation-vision-language-audio-research> (visited on 08/25/2025) (cited on p. 18).
- [7] Jing Ding et al. “Local Temperature Scaling for Probability Calibration”. In: *Proc. ICCV 2021*. Discusses probability calibration with temperature scaling and isotonic regression. 2021. URL: https://openaccess.thecvf.com/content/ICCV2021/papers/Ding_Local_Temperature_Scaling_for_Probability_Calibration_ICCV_2021_paper.pdf (cited on p. 26).
- [8] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: *arXiv preprint arXiv:2010.11929* (2020). Vision Transformer fine-tuning typically uses 3-10 epochs from strong pretrained weights. arXiv: 2010.11929 [cs.CV] (cited on p. 19).
- [9] Cynthia Dwork et al. “Generalization in Adaptive Data Analysis and Holdout Reuse”. In: *arXiv preprint arXiv:1506.02629* (2015). URL: <https://arxiv.org/abs/1506.02629> (cited on p. 24).

- [10] BuiltIn Editors. *Train Test Split: What it Means and How to Use It*. Industry best-practice for 80/20 data splitting. 2025. URL: <https://builtin.com/data-science/train-test-split> (visited on 08/05/2025) (cited on p. 9).
- [11] Facebook AI Research. *ConvNeXt Base (ImageNet-22k, 224x224)*. <https://huggingface.co/facebook/convnext-base-224-22k>. Accessed: 2025-08-13 (cited on pp. 10, 11).
- [12] Tom Fawcett. “An introduction to ROC analysis”. In: *Pattern Recognition Letters* 27.8 (2006), pp. 861–874. DOI: 10.1016/j.patrec.2005.10.010 (cited on p. 13).
- [13] S. Funosas et al. “A Bird Song Detector for improving bird identification in Passive Acoustic Monitoring”. In: *arXiv preprint arXiv:2503.15576* (2024). BirdNET model design and effectiveness for separating bird sounds from background noise. URL: <https://arxiv.org/abs/2503.15576> (cited on p. 26).
- [14] Alexander Gebhard et al. “Exploring Meta Information for Audio-Based Zero-Shot Bird Classification”. In: *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, Apr. 2024, pp. 1211–1215. DOI: 10.1109/icassp48485.2024.10445807. URL: <http://dx.doi.org/10.1109/ICASSP48485.2024.10445807> (cited on p. 4).
- [15] Google Research. *Perch: Bioacoustic Bird Species Classifier*. <https://github.com/google-research/perch>. GitHub repository accessed August 2025. 2025 (cited on p. 26).
- [16] Richard D. Gregory and Arco Van Strien. “Birds as Indicators of Biodiversity”. In: *Ornis Hungarica* 12 (2003). Shows that trends in bird populations reliably reflect wider ecosystem condition, establishing birds as key bio-indicators, pp. 11–24 (cited on p. 1).
- [17] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Sections 7.10–7.10.2; canonical definition of k-fold CV and variance formulas. Springer, 2009. URL: <https://web.stanford.edu/~hastie/ElemStatLearn/printings/ElemStatLearn.pdf> (cited on pp. 17, 18).
- [18] Hyeonuk Nam et al. *Sound Event Detection with FilterAugment*. <https://github.com/frednam93/FilterAugSED/tree/main>. Accessed: 2025-08-20 (cited on p. 22).
- [19] iNaturalist Contributors. *iNaturalist Audio Observations Collection*. <https://www.inaturalist.org>. Citizen-science platform supplying labelled environmental audio clips of birds, mammals, amphibians and insects. 2025 (cited on p. 3).
- [20] Instituto de Investigación de Recursos Biológicos Alexander von Humboldt. *IAvH-CSA Sound Archive*. <https://colecciones.biotaxa.org/IAvH-CSA>. Colombian bioacoustic archive contributing regional species calls to BirdCLEF 2025. 2025 (cited on p. 3).
- [21] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: 1412.6980 [cs.LG]. URL: <https://arxiv.org/abs/1412.6980> (cited on pp. 13, 14).

- [22] Holger Klinck et al. *BirdCLEF+ 2025: Species Identification from Passive Acoustic Monitoring*. <https://www.kaggle.com/competitions/birdclef-2025>. Kaggle Machine Learning Competition, Middle Magdalena Valley, Colombia. 2025 (cited on pp. 3, 25).
- [23] Tom Ko et al. “Audio Augmentation for Speech Recognition”. In: *Interspeech*. 2015, pp. 3586–3589. DOI: 10.21437/Interspeech.2015-711. URL: https://www.danielpovey.com/files/2015_interspeech_augmentation.pdf (cited on p. 18).
- [24] Rohit Kundu. *The Essential Guide to Ensemble Learning*. Accessed 21 August 2025. 2024. URL: <https://www.v7labs.com/blog/ensemble-learning-guide> (cited on p. 23).
- [25] Erin LeDell, Maya Petersen, and Mark van der Laan. “Computationally efficient confidence intervals for cross-validated area under the ROC curve estimates”. In: *Electron J Stat* 9.1 (2015). Methods for confidence intervals in cross-validation, pp. 1583–1601. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4533123/> (cited on p. 17).
- [26] Tsung-Yi Lin et al. *Focal Loss for Dense Object Detection*. 2018. arXiv: 1708.02002 [cs.CV]. URL: <https://arxiv.org/abs/1708.02002> (cited on p. 12).
- [27] Zhuang Liu et al. “A ConvNet for the 2020s”. In: *arXiv preprint arXiv:2201.03545* (2022). URL: <https://arxiv.org/abs/2201.03545> (cited on pp. 1, 2, 9–11, 23).
- [28] Hamza Mahdi, Yasmine Jiang, Samuel Ouellet, et al. “Analysis of Audio Classifier Performance in Clinical Settings with Limited Data”. In: *arXiv preprint arXiv:2402.10100* (2024). Demonstrates transformer models achieving optimal results with just 6 epochs in audio classification with limited data. arXiv: 2402.10100 [cs.SD] (cited on pp. 19, 25).
- [29] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008. ISBN: 9780521865715 (cited on pp. 15, 16).
- [30] MarkovML. *Data Shuffling*. On shuffling and generalization. 2007. URL: <https://www.markovml.com/glossary/data-shuffling> (visited on 08/05/2025) (cited on p. 9).
- [31] Andrea Márquez-Rodríguez, C. Pérez-Granados, and D. Stowell. “A Bird Song Detector for Improving Bird Identification under Overlapping Vocalizations”. In: *Ecological Informatics* (2025). Proposes a detector that first localises bird activity then classifies, reducing false positives caused by mixture of calls and background insects. DOI: 10.1016/j.ecoinf.2025.102001 (cited on p. 2).
- [32] Kaggle competition metrics. *AUC-ROC evaluation metrics*. 2025. URL: <https://www.kaggle.com/code/metric/birdclef-roc-auc> (cited on p. 13).
- [33] Microsoft. *ONNX Runtime for inferencing machine learning models now in preview*. <https://azure.microsoft.com/de-de/blog/onnx-runtime-for-inferencing-machine-learning-models-now-in-preview/>. Describes fast, hardware-independent inference on CPUs and GPUs with ONNX. 2023 (cited on p. 26).

- [34] Romain Mignot et al. “An Analysis of the Effect of Data Augmentation Methods for Musical Genre Classification”. In: *Transactions of the International Society for Music Information Retrieval* (2019). URL: <https://transactions.ismir.net/articles/10.5334/tismir.26> (cited on p. 18).
- [35] Diganta Misra. *Attention Mechanisms in Computer Vision: ECA-Net in PyTorch and TensorFlow*. Overview article with schematic illustration of the ECA module. 2020. URL: <https://blog.paperspace.com/attention-mechanisms-in-computer-vision-ecanet/> (visited on 08/25/2025) (cited on p. 11).
- [36] Meinard Müller. *Discrete Short-Time Fourier Transform (STFT)*. 2015. URL: https://www.audiolabs-erlangen.de/resources/MIR/FMP/C2/C2_STFT-Basic.html (visited on 08/05/2025) (cited on pp. 6, 8).
- [37] Myso. *Post-Processing with Power Adjustment for Low-Rank*. <https://www.kaggle.com/code/myso1987/post-processing-with-power-adjustment-for-low-rank>. Accessed: 2025-08-20. 2025 (cited on pp. 22, 23).
- [38] Myso. *BirdCLEF 2025 5th Place Solution (code and training workflow)*. <https://github.com/myso1987/BirdCLEF-2025-5th-place-solution>. Accessed 20 Aug 2025 (cited on pp. 22, 23).
- [39] Daniel S. Park et al. “SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition”. In: *Proc. Interspeech 2019*. Foundational audio augmentation method widely adopted, including masking and warping. 2019, pp. 2613–2617. DOI: 10.21437/Interspeech.2019-2680. URL: <https://arxiv.org/abs/1904.08779> (cited on pp. 18, 26).
- [40] Thomas Pellegrini and Timothée Masquelier. “Adapting a ConvNeXt model to audio classification on AudioSet”. In: *Proc. Interspeech 2023*. Discusses ConvNeXt architecture, audio feature extraction, training constraints, overfitting, and learning process management. 2023, pp. 4733–4737. DOI: 10.21437/Interspeech.2023-911. URL: https://www.isca-archive.org/interspeech_2023/pellegrini23_interspeech.pdf (cited on p. 25).
- [41] K. J. Piczak. *ESC-50: Dataset for Environmental Sound Classification*. 2015. URL: <https://github.com/karolpiczak/ESC-50> (visited on 08/05/2025) (cited on p. 7).
- [42] David M. W. Powers. “Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness & Correlation”. In: *Journal of Machine Learning Technologies* 2.1 (2011), pp. 37–63 (cited on pp. 15, 16).
- [43] C. J. van Rijsbergen. *Information Retrieval*. 2nd. London, UK: Butterworth-Heinemann, 1979. ISBN: 0408709294 (cited on p. 15).

- [44] Satoshi Saito et al. “Few-shot Bioacoustic Event Detection with Self-supervised Audio Representations”. In: *arXiv* (2024). URL: <https://arxiv.org/abs/2407.10182> (cited on p. 2).
- [45] scikit-learn developers. *Ensembles: Voting — Weighted Average Probabilities (Soft Voting)*. 2025. URL: <https://scikit-learn.org/stable/modules/ensemble.html#weighted-average-probabilities-soft-voting> (visited on 08/21/2025) (cited on p. 24).
- [46] scikit-learn developers. *One-vs-Rest Classification (Multilabel Support)*. Independent per-class modeling for multilabel/one-vs-rest. Jan. 1, 2025. URL: <https://scikit-learn.org/stable/modules/generated/sklearn.multiclass.OneVsRestClassifier.html> (visited on 08/25/2025) (cited on p. 16).
- [47] scikit-learn developers. *Probability Calibration*. Isotonic and sigmoid calibration on held-out data. Jan. 1, 2025. URL: <https://scikit-learn.org/stable/modules/calibration.html> (visited on 08/25/2025) (cited on p. 16).
- [48] David Snyder, Guoguo Chen, and Daniel Povey. “MUSAN: A Music, Speech, and Noise Corpus”. In: *arXiv preprint arXiv:1510.08484* (2015). URL: <https://arxiv.org/abs/1510.08484> (cited on p. 18).
- [49] Dan Stowell et al. “BirdVox-DCASE-20k: A Dataset for Bird Flight Call Detection”. In: *arXiv preprint arXiv:2006.07975* (2020). Discusses sample rate normalization and uniform audio processing for bird audio datasets. URL: <https://arxiv.org/abs/2006.07975> (cited on pp. 1, 2, 9).
- [50] Yuki Tokozume, Yoshitaka Ushiku, and Tatsuya Harada. “Learning from Between-class Examples for Deep Sound Recognition”. In: *ICLR*. 2018. URL: <https://openreview.net/pdf?id=B1Gi6LeRZ> (cited on p. 18).
- [51] Seshuraj Up. *BirdCLEF 2020-2025 All Training+ extra npy Dataset*. Documents down-sampling to 32kHz for uniform preprocessing. 2025. URL: <https://www.kaggle.com/code/seshurajup/birdclef-2020-2025-all-training-extra-npy-dataset/input> (visited on 08/05/2025) (cited on p. 9).
- [52] Ronald E. Walpole et al. *Probability & Statistics for Engineers & Scientists*. Statistical aggregation and confidence intervals formulas for k-fold CV. Pearson, 2011 (cited on p. 17).
- [53] Qilong Wang et al. “ECA-Net: Efficient Channel Attention for Deep Convolutional Neural Networks”. In: *CVPR*. 2020 (cited on pp. 1, 2, 11, 16, 22, 26).
- [54] Wikipedia contributors. *Receiver operating characteristic – Area under the curve*. Accessed: 2025-08-13. 2025. URL: https://en.wikipedia.org/wiki/Receiver_operating_characteristic#Area_under_the_curve (cited on p. 13).

- [55] Xeno-canto Foundation. *Xeno-canto Wildlife Sound Archive*. <https://www.xeno-canto.org>. Community-curated global repository of bird vocalisations; primary audio source for BirdCLEF datasets. 2025 (cited on p. 3).
- [56] Sangdoo Yun et al. “CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features”. In: *ICCV*. 2019. URL: https://openaccess.thecvf.com/content_ICCV_2019/papers/Yun_CutMix_Regularization_Strategy_to_Train_Strong_Classifiers_With_Localizable_Features_ICCV_2019_paper.pdf (cited on p. 18).
- [57] Hongyi Zhang et al. “mixup: Beyond Empirical Risk Minimization”. In: *arXiv preprint arXiv:1710.09412* (2018). URL: <https://arxiv.org/pdf/1710.09412> (cited on p. 18).
- [58] Rui Zhao et al. “Ensemble learning strategy for birdsong recognition under data constraints”. In: *Biodiversity Science* 32.10 (2024). Addresses ensemble methods, blend weight selection, and handling of missing/limited identifiers in birdsong recognition, p. 215. DOI: 10.17520/biods.2024215. URL: <https://www.biodiversity-science.net/EN/10.17520/biods.2024215> (cited on p. 26).